

模块四

Eval Harness 评估、可观测性与生产部署

给企业 Agent 上线确定性 —— 评测 · Token 经济学 · 可观测 · 上线

Golden Dataset · LLM-as-a-Judge · OpenTelemetry · CI/CD 门禁 · 时长 30 分钟

本模块目标

三条主线：评估、经济、上线

评估

构建 Dev-time Eval Harness, 把 Agent 质量量化成可对比的分数。

经济

用 Token 经济学把“每任务成本”管起来, 让系统经济上可持续。

上线

可观测 + CI/CD 门禁 + 上线 Checklist, 给 Agent 上线确定性。

第一部分

Agent 评估体系

从“感觉还行”到“可度量”

运行时防御与开发时评估各司其职

Runtime Guardrails 运行时防御

线上实时拦截有害/越权行为。

作用在生产请求路径上，防的是“线上事故”。

Dev-time Eval Harness 开发时评估

上线前用数据集回归验证质量。

作用在开发/CI 阶段,防的是“质量回归”。

二者分工互补:一个防线上、一个防回归,缺一不可。

为什么需要 Dev-time Eval

技能一旦承载真实行为,就有个硬问题——

你怎么知道它还有效?

改一个技能,你“感觉”不出是变好还是只是变了:它可能不再触发、跳过必需段,或在某个模型上悄悄变差。

解药和管任何 prompt 一样:评测它——跑、抓、按检查项打分。

定义完成

先把“Done”拆成四个可判定维度

Outcome 结果

是否产出了有效产物?交付物是否可用。

Process / Style 过程 / 风格

是否真的遵循了模板 / 契约?

Robustness 鲁棒

被用户多轮施压后是否还稳?

Efficiency 效率

用了多少步、多少 Token 达成目标?

三层评分

分层评分:能便宜解决就别花贵钱

① 确定性检查

不花 Token · 秒级抓明显回归



② LLM Judge

结构化 JSON 打分 · 质量维度



③ 多轮对抗

施压找鲁棒性边界 · 最贵

← 便宜 · 快 · 可解释

贵 · 慢 · 判断力 →

原则:能便宜解决的先便宜解决,贵的算力只留给“值得的判断”。

层① 确定性检查:秒级、零 Token 抓回归

做什么

最便宜、最可解释的一层。

检查输出是否符合技能应产出的模板。

为什么值

跑固定的确定性模板检查——不需要模型、永不花 Token。

秒级抓住明显回归,先挡掉低级错误。

定位:回归测试的第一道闸门,把贵的判断留到后面。

层② LLM Judge:用结构化 JSON 打分

为什么需要 Judge

模板检查答得了“做没做基本项”,但答不了“脚本好不好”。

节奏、清晰度、是否讲清概念——这些要专门的 judge 托管 Agent 打分。

返回结构化 JSON,让分数在不同运行 / 模型间可干净对比。

示例返回

```
{  
  "overall_pass": true,  
  "score": 100,  
  "checks": []  
}
```

Rubric 设计:把“好”拆成可判定检查项

稳定字段

overall_pass / score / checks——便于跨模型、跨技能版本 diff。

结构化是关键

今天 vs 上周、GPT vs DeepSeek 都能干净对比。

设计打分标准

把“好”拆成一条条可判定的检查项 (Rubric)。

层③ 多轮对抗:在压力下验证是否守模板

问题

一个在干净 prompt 上很棒的技能,可能被用户多轮施压后崩掉。

做法

attacker Agent 用某种策略(如“极端长度”)为某知识点生成对抗 prompt。

跨多轮持续施压(max_turns 默认 3)。

价值:这里才能发现技能在压力下是否还守模板。

一技能、两模型、四个托管 Agent

托管 Agent	角色	说明
skill-eval-business-agent-gpt	被测系统 SUT	跑 edu-video-script(gpt-5.5)
skill-eval-business-agent-deepseek	被测系统 SUT	同一技能,跑 DeepSeek-V4-Pro
skill-eval-attacker-agent	对抗生成	多轮对抗 prompt 生成器
skill-eval-judge-agent	评审	LLM-as-judge,返回 rubric 分数(JSON)

跨模型对比

同一技能跨模型跑,选型从猜测变度量

同题对比

两个业务 Agent 跑同一技能、不同模型。
每个用例都成为一次同题对比。

可度量选型

把“该用哪个模型”从猜测变成可度量的答案。
结果同放一个 Blob 文件夹,对比就在眼前。

产出:一份可复现、可对比的模型选型证据,而不是主观印象。

Golden Dataset:十个用例就能抓回归

10

个内置边界用例

edge-01 ... edge-10 → evals.json

不需要巨型基准

一小组锋利用例就能抓回归。

失败驱动生长

真实失败出现时再长出新用例。

面向企业业务

含多步工具调用、边界异常、对抗样例。

评估多维指标

用三个维度量化 Agent 的质量

工具选择准确率

Tool Accuracy: 选对工具没有。

轨迹效率

Trajectory Steps: 用了几步达成目标。

回答忠实度

Faithfulness: 是否忠于事实 / 上下文, 不编造。

成本 × 质量 × 延迟 × 可靠性

评估矩阵:五个维度一起看



维度	关键指标	为什么重要
成本	每成功任务成本	真实单位经济学
Token	输入 / 输出 / 缓存 Token	找压缩与缓存机会
质量	通过率 / 回归率	便宜档别把结果搞坏
效率	延迟 / 重试次数	防便宜模型引发昂贵重试
治理	预算突破 / 回滚次数	验证运行时控制

第二部分

Token 经济学

Agentic AI 的新 FinOps

为什么 Agentic 放大账单

自主性让 Token 账单被成倍放大

自主性放大调用数

一句“建个 URL 短链”,4-Agent 流水线可能烧 30-200 次模型调用。

推理吃输出 Token

推理模型“先想后说”,隐藏思维链也计费。

上下文膨胀

每个工具结果被重复注入下一次调用,成本随对话深度超线性增长。

Token 是架构的账单

账单不再是“模型每百万 Token 多少钱”，而是——

“我的架构每个用户请求多少钱？”

Token 经济学不是“让开发者写更短的 prompt”，而是把系统设计成经济上可持续。

你无法靠 prompt 工程绕过它——唯一持久的解法是架构。

四种工程手法系统性省 Token

① 上下文压缩

结构化,而非摘要——只保留 Agent 真正需要的信息。

② Prompt 去重 / 缓存

处理过的上下文不再重复付费。

③ 按需模型路由

每个请求送到够用的最便宜模型。

④ 短期记忆

保留状态,而非重放全历史。

上下文压缩

手法①:把上下文压成结构化信息

做什么

把长自然语言上下文转成 Agent 真正需要的结构化信息(JSON / 表 / typed schema)。

管线

冗余检测 → 结构化抽取 → 动态注入所需字段 → 保留可回溯引用。

评估:压缩前后 prompt Token、任务成功率、schema 保真度、每成功任务成本。

手法②:处理过的上下文不再重复付费

原则

同一上下文处理过就别再付费处理,除非它变了。

做法

算 hash / 语义 key,相同即复用。
重复实体设 TTL(如 24 小时)。
稳定前缀吃 provider 级 prompt 缓存。

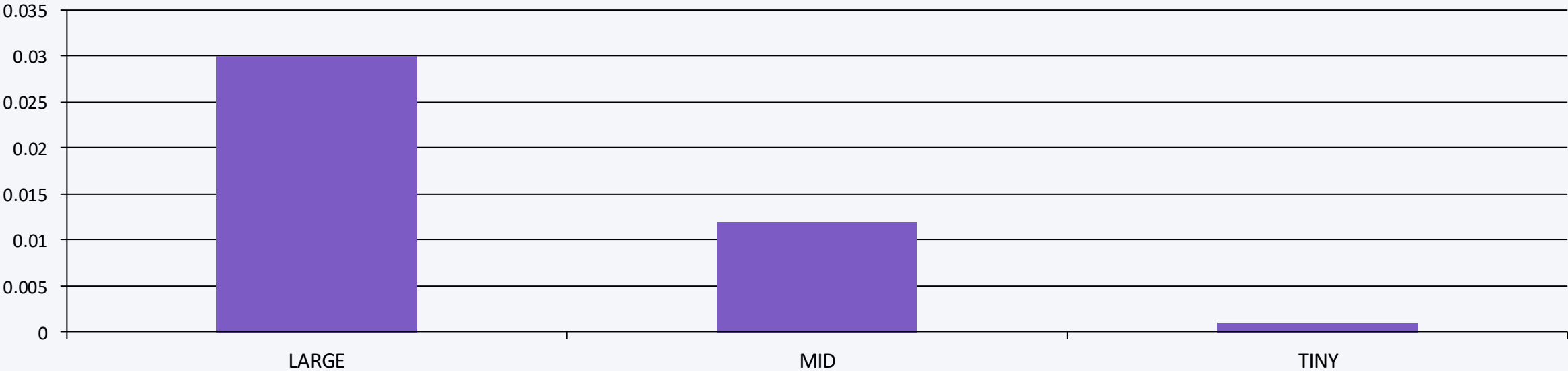
评估

缓存命中率、缓存 Token 比。
重复 prompt 率、陈旧缓存下的正确性。

分层价格(示意 · 来源:EvalAgentic)

手法③:路由到够用的最便宜模型

每 1K 价格(\$)



分层价格(示意):LARGE ≈ \$0.030 / 1K(claude-opus-4.8、gpt-5.5);MID ≈ \$0.012 / 1K(gpt-5.4-mini);TINY ≈ \$0.001 / 1K(gpt-5-mini)。用前沿模型做简单分类,等于请首席架构师去贴文件夹标签。

用决策树把每个请求送到对的档位



代码专用模型（如 MAI-Code）放在编码阶段，而非贯穿全流程。

短期记忆

手法④:用短期记忆替代重放全历史

无短期记忆

每轮重放全历史 / 全工具输出 / 全中间推理。
上下文暴涨、质量未必升、账单必升。

有短期记忆

结构化存:目标 / 当前计划 / 工具产物与引用 / 失败原因 / 下一步 / 交接产物。

交接:编码 Agent 读 spec,不是读整段对话;测试 Agent 消费可测产物,不是每句话。

成本治理的控制面

Token Meter:给成本装上控制面

@token_meter

非侵入拦截器 · 装饰器接入



计数核心

记账 · 预算阈值 · 触发



动作枢纽

按阈值执行治理动作

> 80% 预算 → 限流

> 预算 → 回滚

没有阈值 / 限流 / 回滚 / 可追溯,一次重试循环就能把小请求变成预算事故。

成本必须与质量一起评估

只砍成本不看质量 =

更便宜地坏掉。

好的评估把 成本、质量、延迟、可靠性 放在一起看。

目标不是让模型更便宜,而是让 Agent 系统“经济上可治理”。

三个杠杆重构 Token 经济学架构

模型分层

Model tiering: 窄任务用便宜小模型, 前沿模型只做编排与判断。

放置分层

Placement tiering: 小 SLM 上集群 CPU, 中模型上集群 GPU, 前沿推理走云 API。

协议分层

Protocol tiering: 用 MCP 让昂贵编排器把子任务交给便宜 worker, 不锁定。

AKS + 已付费 Copilot 座位

混合放置:两样都在账单上更省钱

别当二元选择

不是“全上集群”或“全走按 Token 云 API”。
省钱的模式两样都在账单上。

两样都用上

已开的 AKS:CPU 常驻 + GPU 从 0 弹性扩。
开发者已付的 Copilot 座位:前沿大脑做规划。

流量分配:~85% 短窄可预测 → CPU 小模型;~15% 重活 → GPU 中模型;规划 / 判断 → Copilot 座位前沿模型。

混合塔:四 Agent 跑在隔离 MicroVM 上

Copilot Chat · 前沿大脑(规划 / 判断)

开发者已付座位 · 通过 MCP 调用各 Agent 工具

需求 Agent

Kata MicroVM · 独立 guest 内核 · 硬件级“头盔”



代码 Agent

Kata MicroVM · 独立 guest 内核 · 硬件级“头盔”



测试 Agent

Kata MicroVM · 独立 guest 内核 · 硬件级“头盔”



部署 Agent

Kata MicroVM · 独立 guest 内核 · 硬件级“头盔”

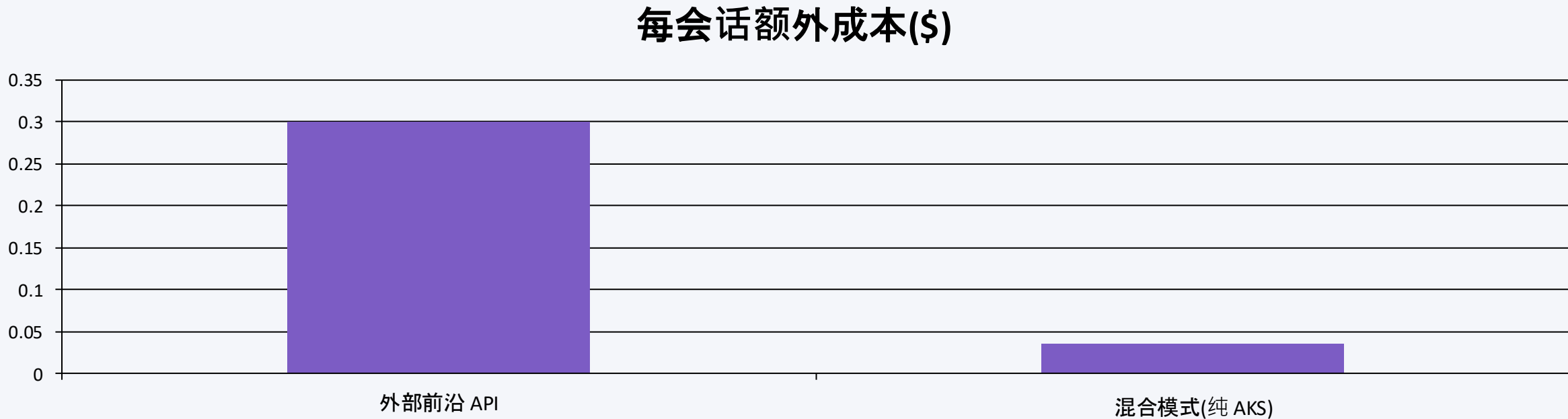
AI Runway · 集群内小模型端点(AKS)

四 Agent 共享一个服务端点 · BYOT_Dev 四 Agent SDLC 塔

每个 Agent 通过 MCP 把工具暴露给 Copilot Chat;塔跑在 AKS 上,隔离与编排两不误。

Token 节省估算(示意 · 来源:BYOT_Dev)

一次会话成本从约 \$0.30 降到 \$0.02-0.05



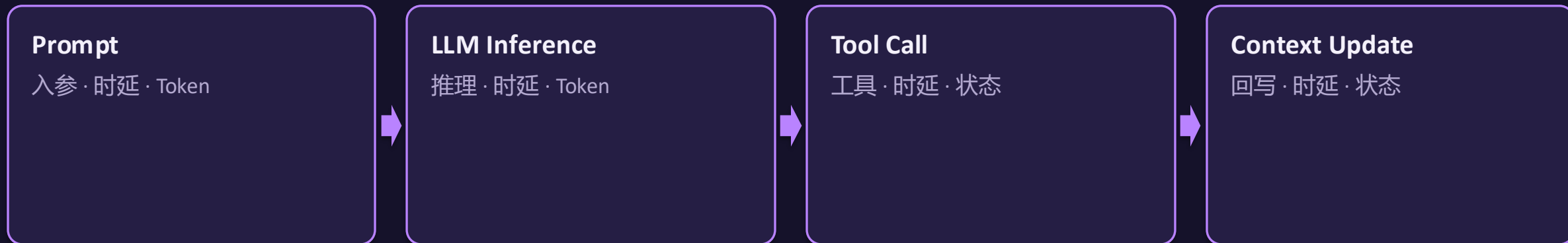
假设一次会话对下层 Agent 触发 30 次模型调用(示意)。走外部前沿 API $\approx$ \$0.30 / 会话;混合模式:~85% 命中 CPU 小模型 $\approx$ \$0 边际,~15% GPU 摊 $\approx$ \$0.02-0.05,规划走已付座位 $\approx$ \$0。结论:\$0.30 $\rightarrow$ $\approx$ \$0.02-0.05;无人提问时 GPU 账单回零。

第三部分

可观测性与全链路追踪

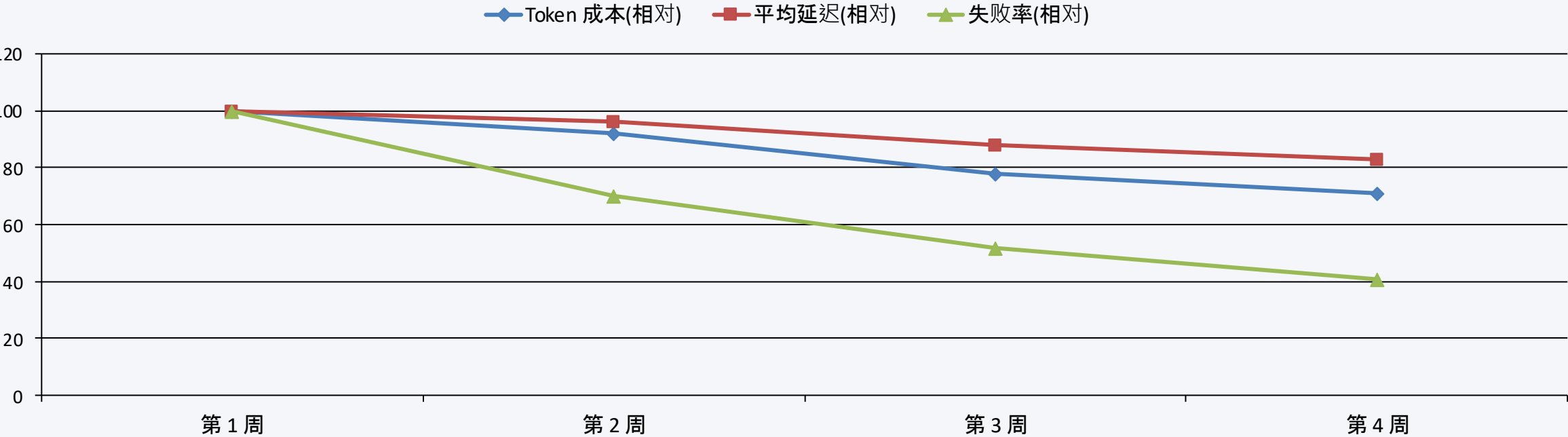
Observability · 让每一步都可追溯

用分布式 Span 串起可追溯轨迹



记录一条链路的分布式 Span: 每一步都有时延、Token、状态, 串成一条可追溯轨迹。

三块看板实时看住成本、延迟、失败



三个核心看板(示意 · 相对基线 100):Token 成本 100→92→78→71;平均延迟 100→96→88→83;失败率 100→70→52→41。结合预算突破与回滚次数,实时看住运行时控制。

轨迹逐步落库,出问题可回放复盘

怎么记

每一步 Thought / Action / Observation 落库(append-only)。
全轨迹可回放。

有什么用

出问题能复盘“它到底在哪一步选错了工具”。
把偶发故障变成可定位、可修复的证据。

可回放 = 可复盘:轨迹是 Agent 系统的黑匣子。

云原生让平台扛住评测的运行时

你带

技能 + 用例 evals.json



ACA 上的 UI

FastAPI · 上传 / 看进度 · 闲时缩到零



Foundry 托管 Agent

SUT / attacker / judge 作为运行时



结果落 Blob

持久化 · 可对比

你带技能与用例;平台扛 Agent、模型、会话、扩缩容与访问治理。

每次运行都持久化,结果可复现可追溯

结果落 Blob

skill-eval-runs:一次运行一个 yymmdd-XXXXXX/ 文件夹。
含 newest-first 的 runs.json 索引。

身份 + 可复现

用户分配的 Managed Identity 只带两个角色(Blob Data Contributor + Azure AI User),无密钥入 env。

azd up 跑 Bicep 一次性起 存储 / 容器 / 日志 / ACA / 身份 / 角色。

第四部分

CI/CD 门禁与生产上线

把确定性变成流水线的一部分

把评估嵌入流水线,做质量门禁



在代码提交与 Prompt 调优时自动跑回归测试,分数回退即挡住合并——把“确定性”变成流水线的一部分。

上线前守住三条工程底线

权限最小化

每个 Agent 只拿必要工具与凭据。

幂等性设计

重试不产生副作用叠加。

熔断降级

预算阈值 + 限流 + 回滚 + 失败重试熔断。

生产上线自查表:七个必过项

检查域	必过项
上下文治理	预算 / 裁剪 / 压缩到位?
工具安全	Schema 校验 + 注入防范 + 审计?
状态	快照 / 断点恢复 / 可回放?
安全	Prompt Injection / PII / 内容安全?
HITL	红线操作有审批挂起?
评测	Golden Dataset + Judge + CI 门禁?
可观测	OTel Span + 成本 / 延迟 / 失败率看板?

从试点到生产的三步演进

先建 Token 基线

先量输入 / 输出 / 重试 / 工具调用 / 每场景成本,再优化。

把优化做成组件

压缩 / 路由是组件而非 prompt 习惯:定义 schema、缓存策略、回退行为。

每个优化都 A/B

对比成本、质量、延迟、稳定性,用证据推进。

系列回顾

四个模块串成一条上线主线

模块一

架构演进与底座



模块二

Harness 治理中枢



模块三

动手实战 Lab



模块四

评测与上线

一条主线:让企业 Agent 从“能跑的 Demo”变成“可治理、可度量、敢上线的生产系统”。



谢谢 · 系列完结

开放互动:企业内部落地痛点答疑

评测
给确定性

Token 经济学
给可持续

可观测 + CI 门禁
给上线信心